

Design Patterns

DE52

Yazan MUALLA

CIAD, Univ. Bourgogne Franche-Comté, UTBM,
France

- o Introduction
- o Use of Design Patterns
- o Detailed Discussion and Examples
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

Object-Oriented Modelling

- Object-oriented modelling methods emphasize the **design notation**
- But it is more than just drawing diagrams, it is about designing the best solution
- Most powerful reuse is the **design reuse** (match problem to design experience)
- Object-oriented modelling systems exhibit *recurring design structures* that promote:
 - Abstraction
 - Flexibility
 - Modularity
 - Elegance
 - Efficiency
 - ...

Beginning of Patterns

“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over.”

Christopher Alexander, A Pattern Language, 1977

Why Study Patterns?

- Reuse tried and proven solutions
 - No need to reinvent the wheel
 - Provide a head start and save time
 - Increase *efficiency*
- Establish common terminology
 - Increase inter-relations benefits in shared work
- Provide a higher-level perspective
 - Free us from dealing with the details too early
- Most design patterns make software systems more *modifiable*
 - We are using time-tested solutions
- Make software systems easier to update (more *maintainable*)
- Help increase the understanding of basic **object-oriented design principles**



- o Introduction
- o **Use of Design Patterns**
- o Detailed Discussion and Examples
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

Design Patterns

- A technique to repeat the designer success and intensify the design experience
- **(Problem, Solution) pair** that abstracts a recurring design structure
- Name & specify the design structure explicitly
- Four Basic Parts:
 - Name
 - Problem
 - Solution
 - Uses & trade-offs
- UML is used for describing Design Patterns

Design Patterns Goals

- Codify good design
 - Generalized experience
 - Aid to novices & experts alike

- Give *design structures* explicit names
 - Common vocabulary
 - Reduced *complexity*
 - Greater *expressiveness*
 - Improve documentation

- Facilitate *refactoring*
 - Patterns are interrelated
 - Additional *flexibility*

Refactoring

- The **process of restructuring the code** in a series of small and semantics-preserving transformations without changing its external behavior
- It improves *nonfunctional* requirements of the software:
 - Improve source-code *readability* and reduce *complexity*
 - Improve source-code *maintainability*
 - Create a more expressive internal architecture or object model to improve *extensibility*
- **Extreme Programming** is a form of Agile Programming that emphasizes refactoring
- **Unit testing** is an essential component of Extreme Programming, and it is testing units (like classes) in isolation

Object-Oriented Design Patterns

- **Creational patterns:** deal with the process of class/object *creation*

• Abstract Factory • Builder	• Factory Method • Prototype	• Singleton
--	--	---

- **Structural patterns:** deal primarily with the *static composition and structure* of classes and objects

• Adapter • Bridge • Composite	• Decorator • Façade	• Flyweight • Proxy
--	--	---

- **Behavioral patterns:** deal primarily with the *dynamic interaction* among classes and objects

• Chain of Responsibility • Command • Interpreter • Iterator	• Mediator • Memento • Observer • State	• Strategy • Template Method • Visitor
--	---	--

book (*Gang of Four Design Patterns*)

Object-Oriented Design Patterns (cont.)

		<i>Purpose</i>		
		Creational	Structural	Behavioral
Scope	Class	<u>Factory Method</u>	<u>Adapter (class)</u>	Interpreter <u>Template Method</u>
	Object	Abstract Factory Builder Prototype <u>Singleton</u>	Adapter (object) Bridge <u>Composite</u> Decorator Flyweight Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento <u>Observer</u> State Strategy Visitor

- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - **Object Behavioral Patterns**
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

Observer (object behavioral)

- **Problem:** Need to notify a changing number of objects (observers) about an update in an entity (subject)
- **Solution:** Define a **one-to-many dependency** so that when one entity updates its state, all its dependents (observers) are notified automatically
- **Uses:**
 - A change to one entity requires informing others
- **Consequences:**
 - + *Modularity*: subject & observers may vary independently
 - + *Extensibility*: can define & add any number of observers
 - + *Customizability*: different observers offer different views of the subject. E.g., Online stores
 - Unexpected or heavy updates: observers may be overwhelmed

Observer (object behavioral)

- Whenever the subject undergoes a change in its state, it notifies all of its registered observers
- Upon receiving a **notification** from the subject, each of the observers queries the subject to **synchronize** its state with that of the subject's (Pull model!)
- The **subject** should provide an **interface for registering/unregistering and for notifications**
- **Observers** should provide an **interface for receiving notifications** from the subject

Observer (object behavioral)

- Publisher-subscriber, one of the following two must be true:
 - In the **pull model**: The subject should provide an interface that enables observers to query the subject for the required state information to update their state
 - In the **push model**: The subject should send the state information that the observers may be interested in
- After applying the Observer pattern, **different observers can be added dynamically** without requiring any changes to the Subject class

Observer (object behavioral)

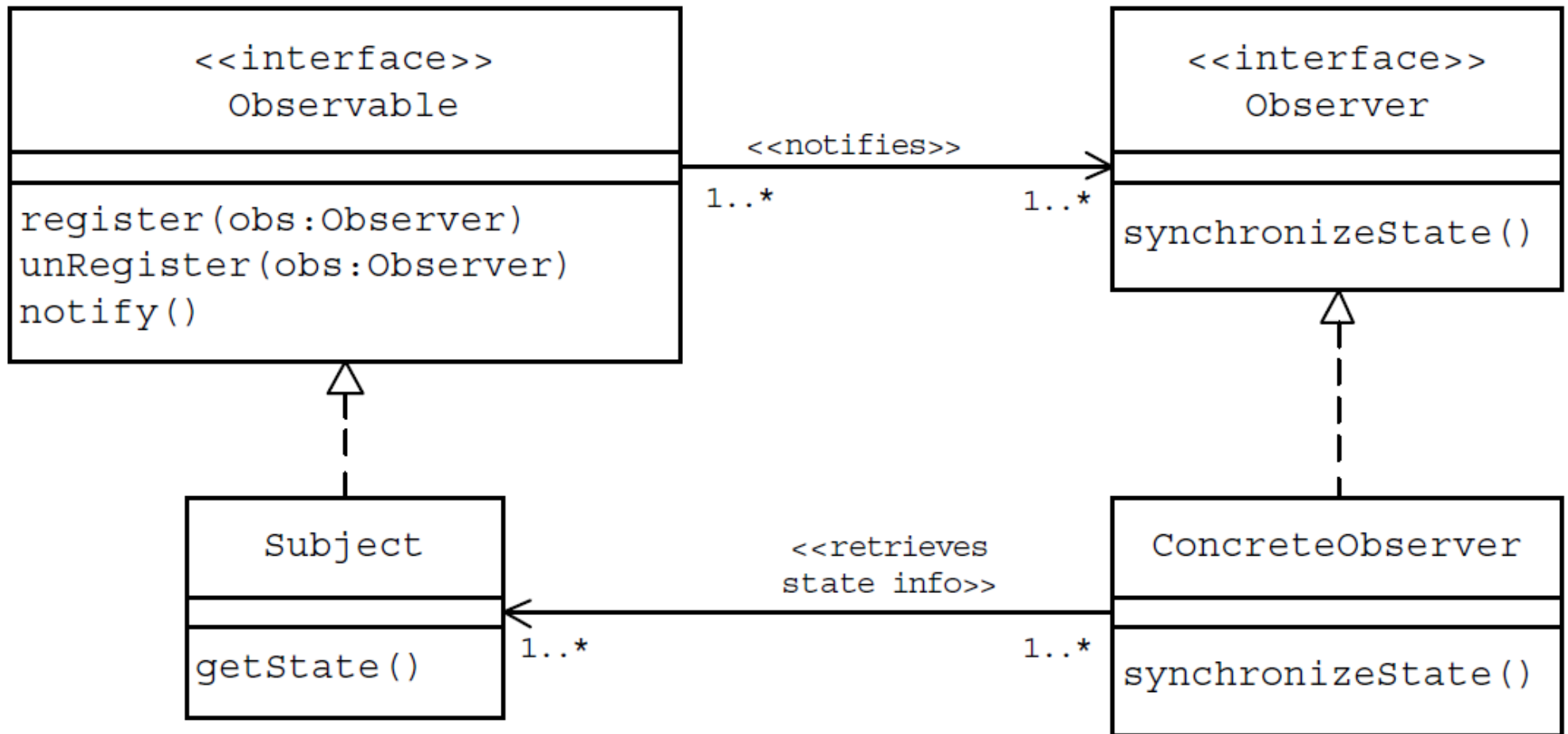
■ Implementation:

- Subject-observer mapping
- Update protocols: the push & pull models

■ Examples:

- Smalltalk Model-View-Controller (MVC)
- Pub/sub middleware (e.g., CORBA Notification Service, Java Messaging Service)
- Mailing lists
- Social media followers

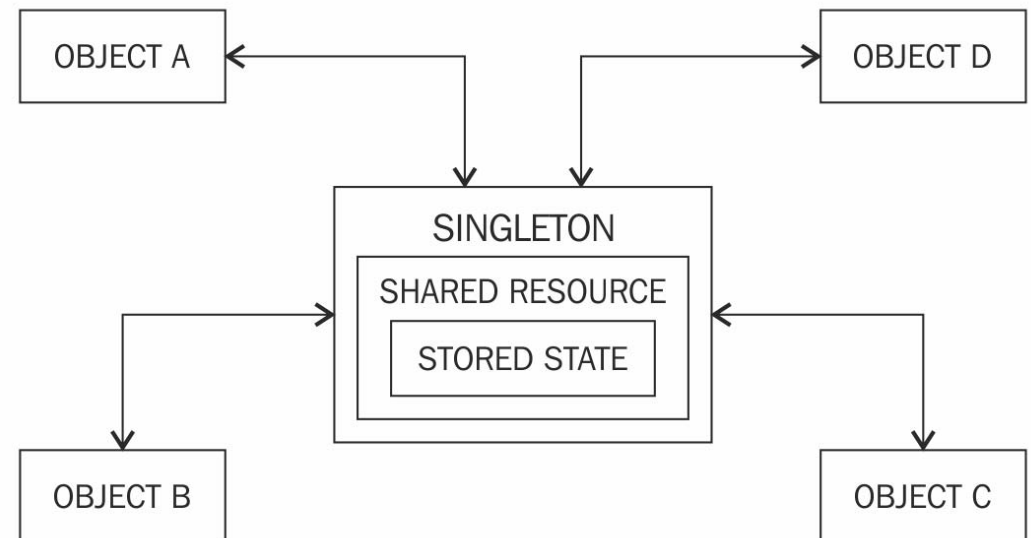
Observer (object behavioral)



- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - Object Behavioral Patterns
 - **Object Creational Patterns**
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

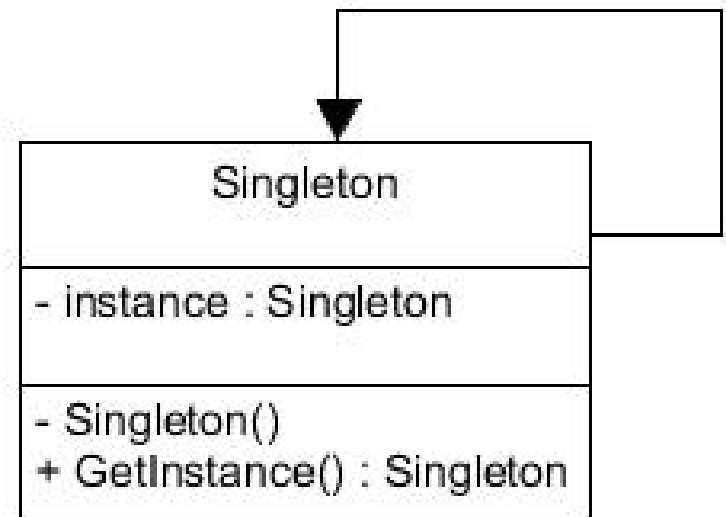
Singleton (object creational)

- **Problem:** Need to have **one and only one instance of a given class** during the lifetime of an application
- **Solution:** Provides a **controlled object creation mechanism** to ensure that only one instance of a given class exists
- **Examples:**
 - AudioStream
 - A single database connection object



Singleton (object creational)

```
class Singleton {  
    private static final Singleton instance = new  
Singleton();  
  
    // don't let Java give you a default public constructor  
    private Singleton() { }  
  
    public static Singleton GetInstance() {  
        return instance;  
    }  
    ...  
}
```



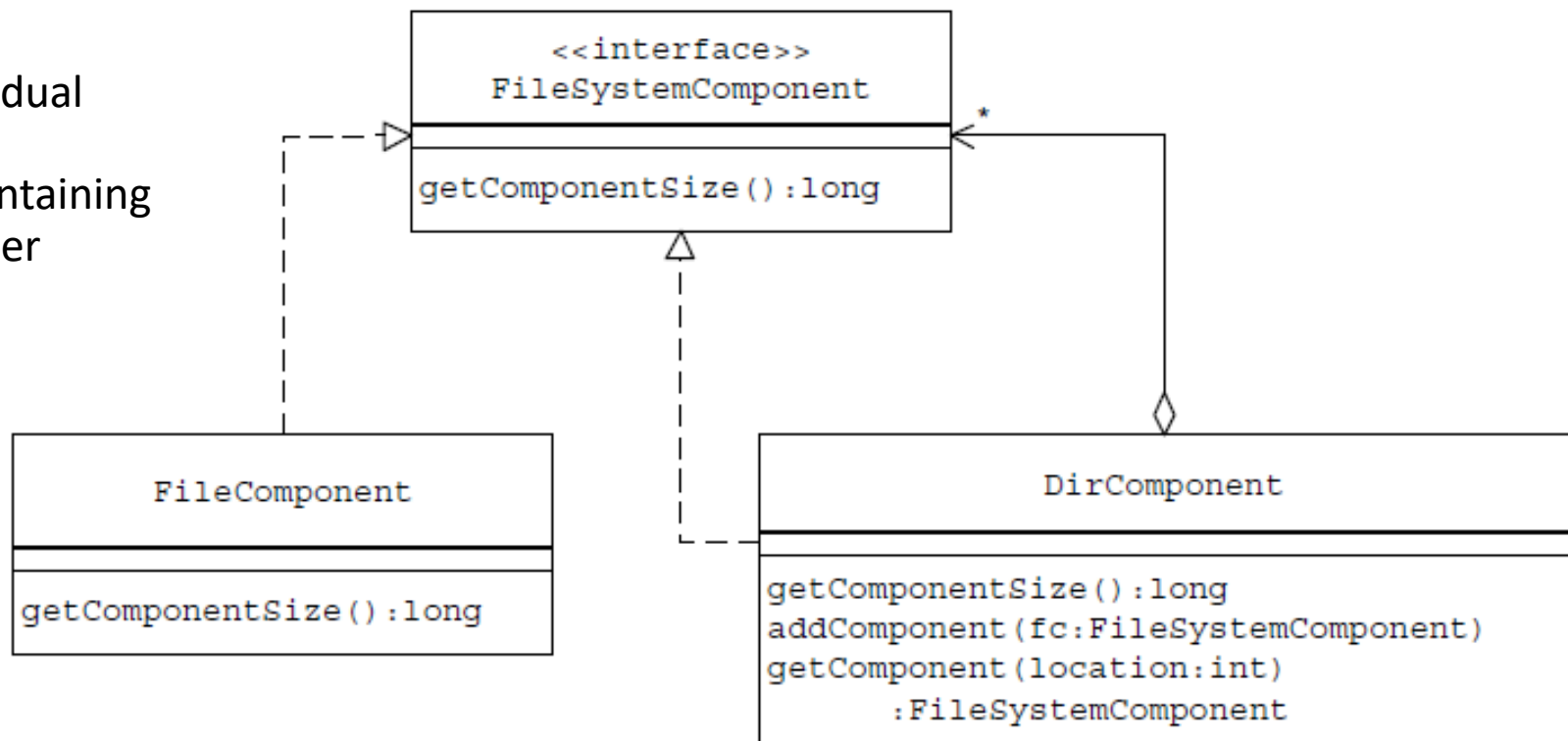
- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - Object Behavioral Patterns
 - Object Creational Patterns
 - **Object Structural Patterns**
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

Composite (object structural)

- **Problem:** Need to treat individual objects & multiple, recursively-composed objects uniformly
- **Solution:** The object **inherits** the abstract class or **realizes** the interface that is composed of or aggregates the object
- **Uses:**
 - No distinction between individual & composed elements
 - Objects in structure can be treated uniformly
- **Consequences:**
 - + *Uniformity*: treat components the same regardless of complexity
 - + *Extensibility*: new component subclasses work wherever old ones do
 - _ Ambiguity

Composite (object structural)

- You want to treat **uniformly**
 - files (individual objects)
 - folders (containing files or other folders)



Composite (object structural)

■ Implementation:

- Do components (children) know their composites (parents)?
- Uniform interface for both components & composites?
- Responsibility for deleting children

■ Examples:

- Directory structures in UNIX & Windows
- Naming Contexts in CORBA
- MIME types in SOAP

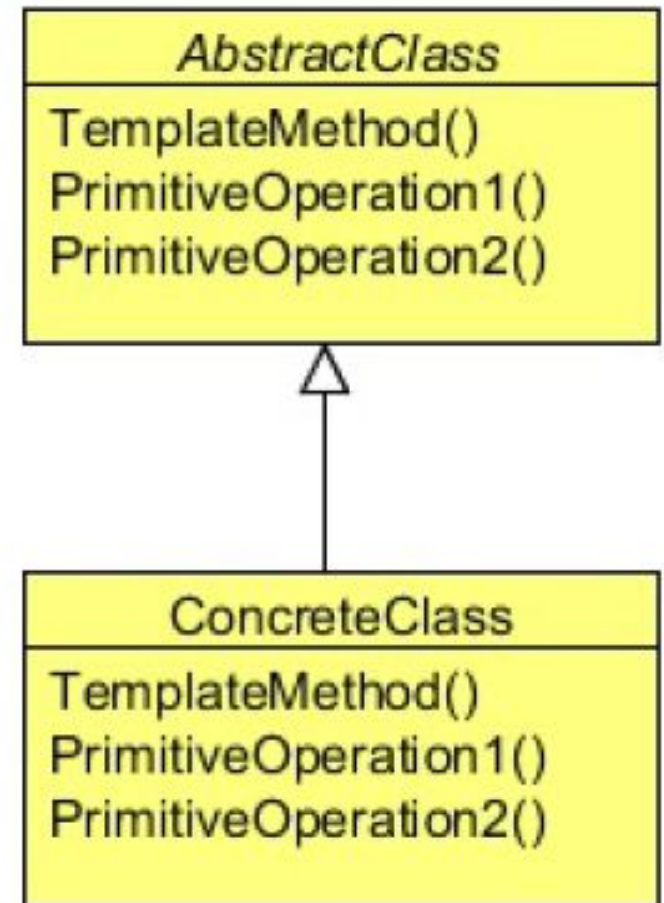
- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - **Class Behavioral Patterns**
 - Class Creational Patterns
 - Class Structural Patterns
- o Summary

Template Method (class behavioral)

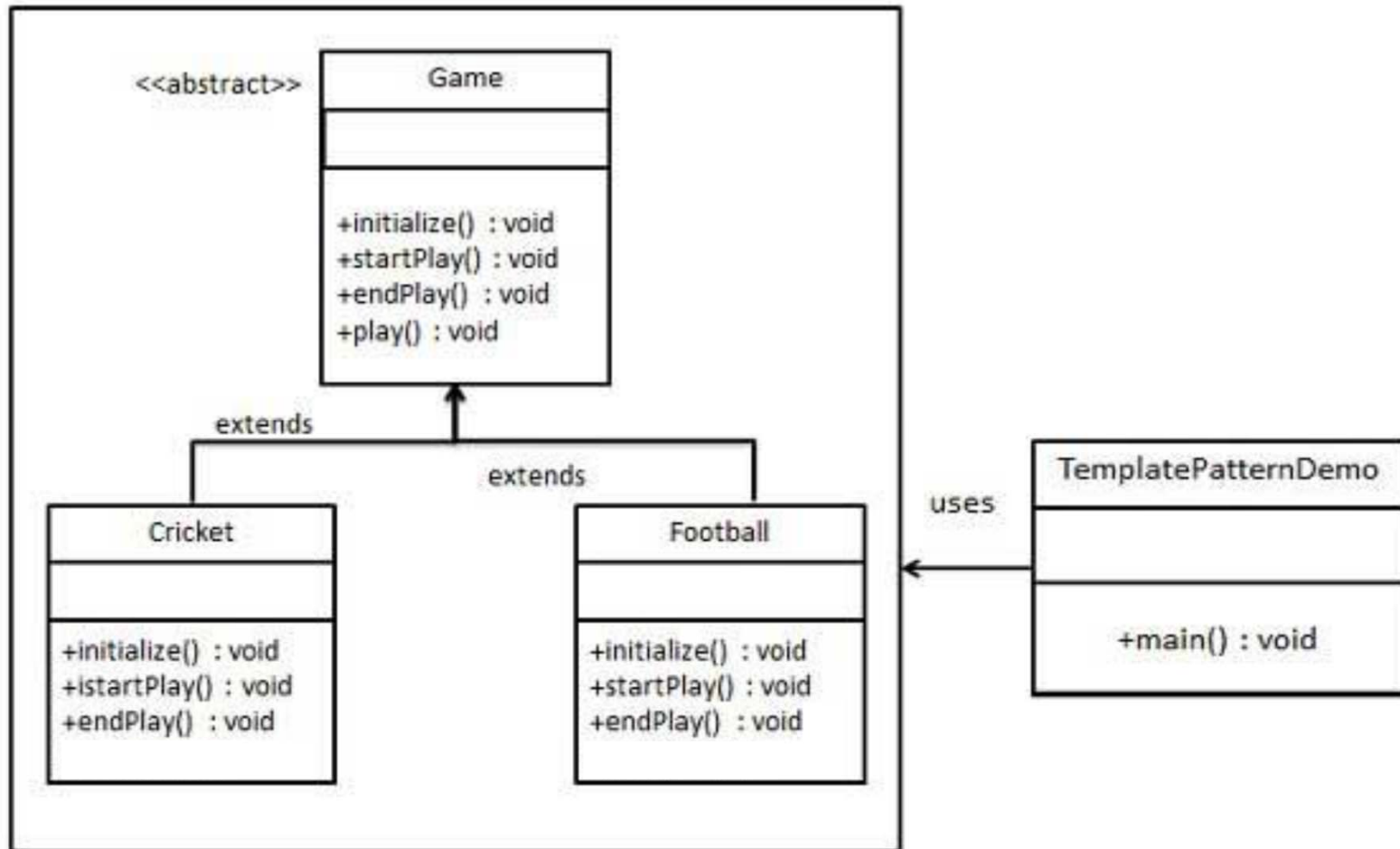
- **Problem:** Having similar implementations of the same functionality with some variant parts
- **Solution:** Provide an **abstract class** that defines way(s)/template(s) to execute its methods. Its subclasses can override the method implementation as per need but the invocation is to be in the same way as defined by the abstract class
- **Uses:**
 - When there is an algorithm that could be implemented in multiple ways, the template method pattern enables keeping the outline of the algorithm in a separate method (Template Method) inside a class (Template Class), leaving out the specific implementations of this algorithm to different subclasses
 - Keep the invariant part of the functionality in one place and allow the subclasses to provide the implementation of the variant part

Template Method (class behavioral)

- Defines abstract primitive operations that concrete subclasses use to implement steps of an algorithm
- Implements a template method defining the skeleton of an algorithm.



Template Method (class behavioral)



Template Method (class behavioral)

```
abstract class Game {  
    // Template method  
    public final void play() {  
        initialize();  
        startPlay();  
        endPlay();  
    }  
  
    // Steps to be implemented by  
    subclasses  
    public abstract void initialize();  
    public abstract void startPlay();  
    public abstract void endPlay();  
}
```

```
class Football extends Game {  
    @Override  
    public void initialize() {  
        System.out.println("Football Game Initialized! Start  
playing.");  
    }  
  
    @Override  
    public void startPlay() {  
        System.out.println("Football Game Started. Enjoy  
the game!");  
    }  
  
    @Override  
    public void endPlay() {  
        System.out.println("Football Game Finished!");  
    }  
}
```

Template Method (class behavioral)

```
public class Main {  
    public static void main(String[] args) {  
        Game game = new Football(); // Polymorphism in action  
        game.play(); // Template method  
    }  
}
```

Output:

Football Game Initialized! Start playing.
Football Game Started. Enjoy the game!
Football Game Finished!

- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - **Class Creational Patterns**
 - Class Structural Patterns
- o Summary

Factory Method (class creational)

- **Problem:** Need to **reduce coupling** between client objects and class hierarchy structure
- **Solution:** Define an interface to choose the right class from the hierarchy structure as per some criteria
- **Uses:**
 - A class can't anticipate the class of objects it must create
 - A class wants its subclasses to specify the objects it creates
 - Classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate

Factory Method (class creational)

■ Examples:

- XML files handlers
- Image creator:

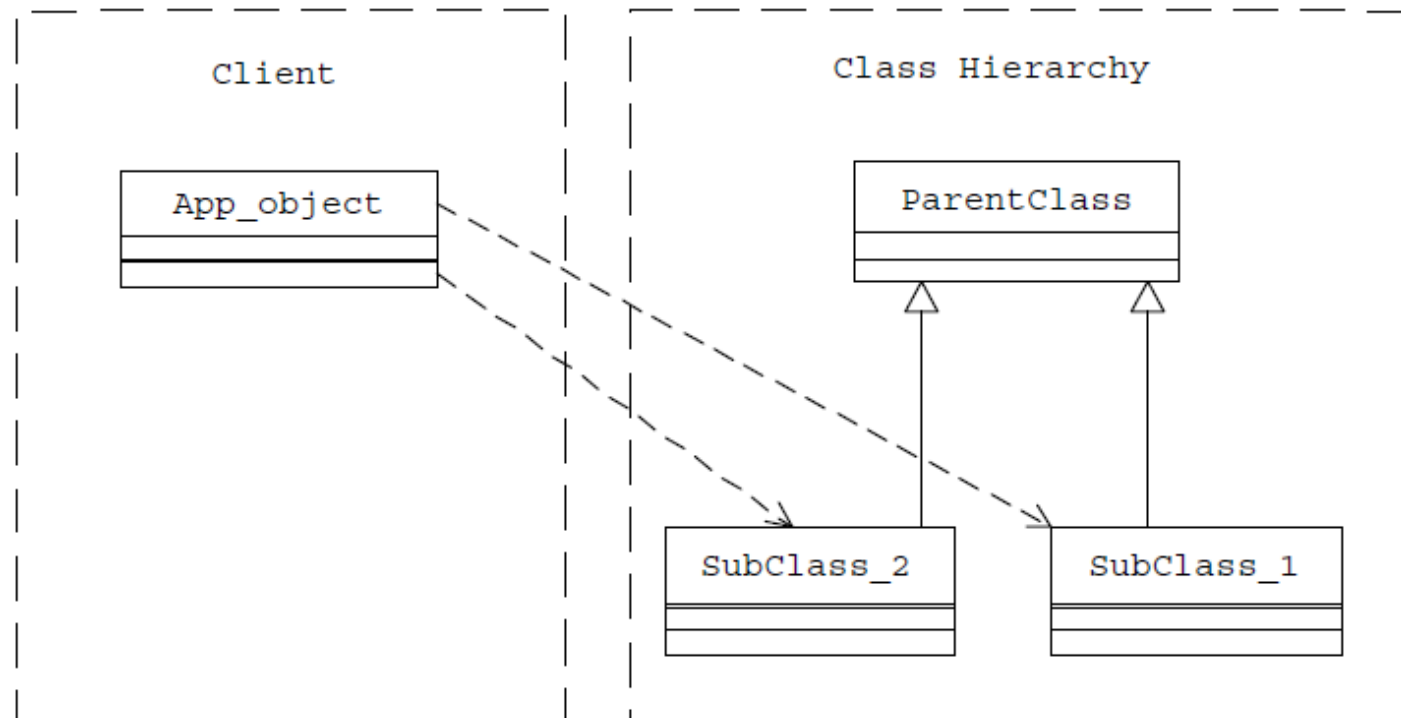
```
Image createImage (String ext) {  
    if (ext.equals("gif")) return new GIFImage();  
    if (ext.equals("jpg")) return new JPEGImage();  
    ...  
}
```

?!

Image example drawbacks: High coupling, Dependency, Readability, Scalability

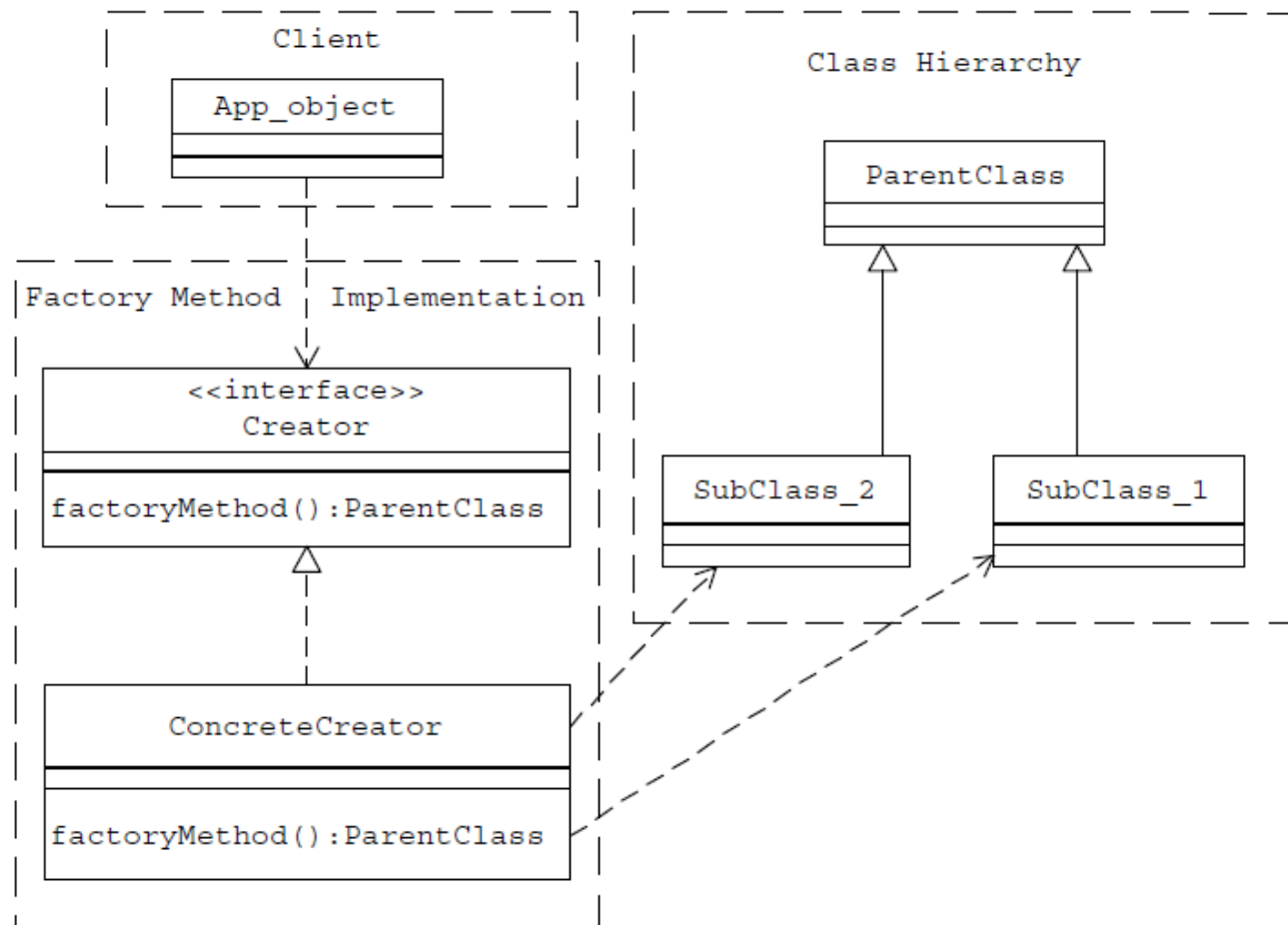
Factory Method (class creational)

- Problem



Factory Method (class creational)

- Solution



Factory Method (class creational)

```
interface Vehicle {  
    void start();  
}
```

```
class Car implements Vehicle {  
    @Override  
    public void start() {  
        System.out.println("Car started!");  
    }  
}
```

```
class Bike implements Vehicle {  
    @Override  
    public void start() {  
        System.out.println("Bike started!");  
    }  
}
```

Factory Method (class creational)

```
abstract class VehicleFactory {  
    // Factory method  
    public abstract Vehicle createVehicle();  
}  
  
class CarFactory extends VehicleFactory {  
    @Override  
    public Vehicle createVehicle() {  
        return new Car(); // Creates a Car object  
    }  
}  
  
class BikeFactory extends VehicleFactory {  
    @Override  
    public Vehicle createVehicle() {  
        return new Bike(); // Creates a Bike object  
    }  
}
```

Factory Method (class creational)

```
public class Client {  
    public static void main(String[] args) {  
        VehicleFactory factory;  
  
        // Client decides which factory to use  
        factory = new CarFactory();  
        Vehicle car = factory.createVehicle();  
        car.start();  
  
        factory = new BikeFactory();  
        Vehicle bike = factory.createVehicle();  
        bike.start();  
    }  
}
```

Output:

Car started!

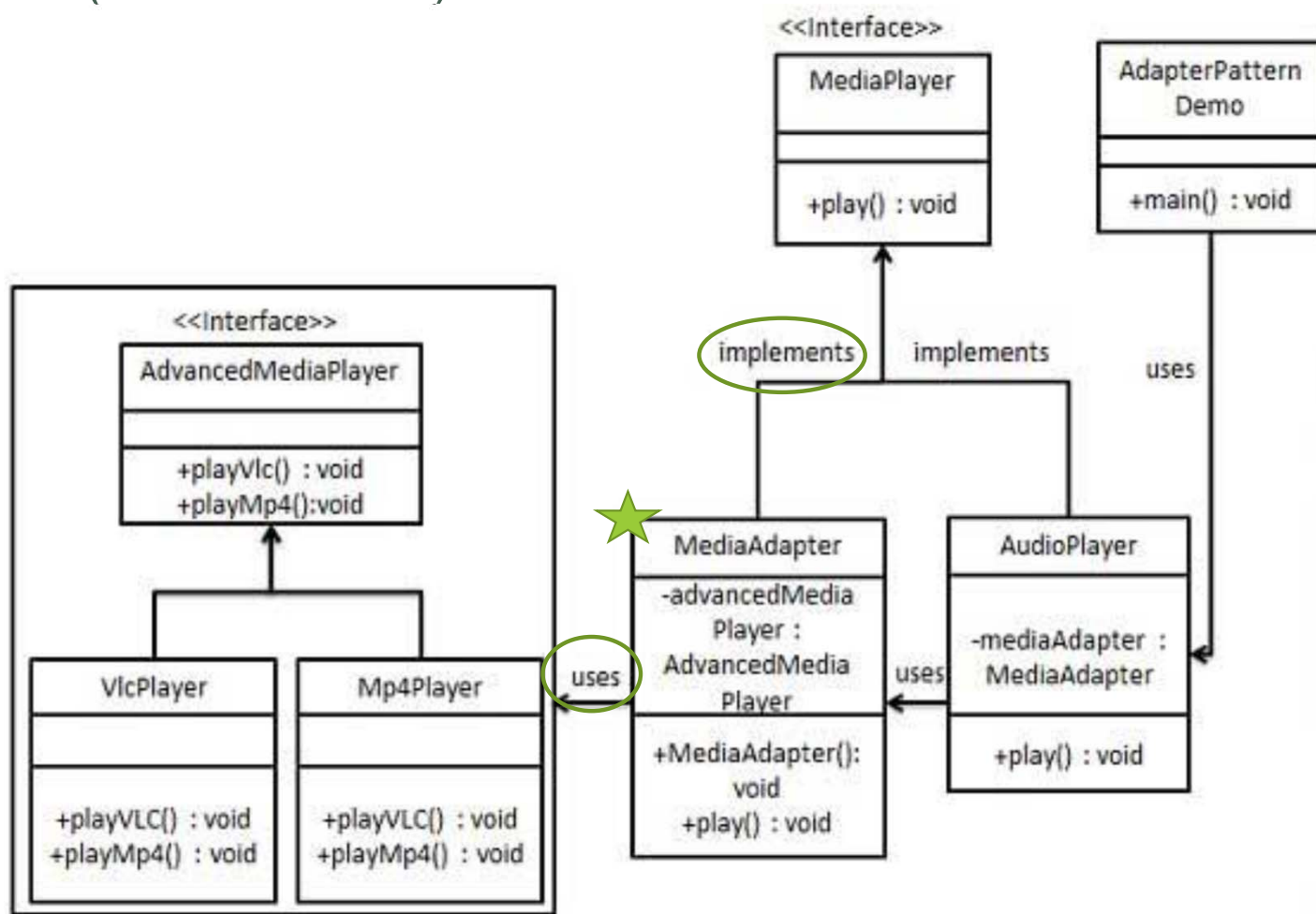
Bike started!

- o Introduction
- o Use of design patterns
- o **Detailed Discussion and Examples**
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - **Class Structural Patterns**
- o Summary

Adapter (class structural)

- **Problem:** Need to fill the gap between two interfaces or **connect legacy code with new code** with no changes to be made between the two parts
- **Solution:** Define a bridge between the two parts
- **Uses:**
 - This pattern works as a bridge between **two independent and incompatible entities**
 - This pattern involves a single class which is responsible to join functionalities of entities
- **Examples:**
 - Card reader
 - Audio player adapter

Adapter (class structural)



Adapter (class structural)

```
public interface MediaPlayer {  
    void play(String audioType);  
}
```

```
public interface AdvancedMediaPlayer {  
    void playVlc();  
    void playMp4();  
}
```

Adapter (class structural)

```
public class VlcPlayer implements AdvancedMediaPlayer {  
    @Override  
    public void playVlc() {  
        System.out.println("Playing vlc file");  
    }  
    @Override  
    public void playMp4(String fileName) {  
        // do nothing  
    }  
}
```

```
public class Mp4Player implements AdvancedMediaPlayer {  
    @Override  
    public void playVlc() {  
        // do nothing  
    }  
    @Override  
    public void playMp4(String fileName) {  
        System.out.println("Playing mp4 file");  
    }  
}
```

Adapter (class structural)

```
public class MediaAdapter implements MediaPlayer {
    AdvancedMediaPlayer advancedMediaPlayer;

    public MediaAdapter(String audioType) {
        if(audioType.equalsIgnoreCase("vlc")) {
            advancedMediaPlayer = new VlcPlayer();
        } else if(audioType.equalsIgnoreCase("mp4")) {
            advancedMediaPlayer = new Mp4Player();
        }
    }

    @Override
    public void play(String audioType) {
        if(audioType.equalsIgnoreCase("vlc")) {
            advancedMediaPlayer.playVlc();
        } else if(audioType.equalsIgnoreCase("mp4")) {
            advancedMediaPlayer.playMp4();
        }
    }
}
```

Adapter (class structural)

```
public class AudioPlayer implements MediaPlayer {
    MediaAdapter mediaAdapter;

    @Override
    public void play(String audioType) {
        if(audioType.equalsIgnoreCase("mp3")) {
            System.out.println("Playing mp3 file");
        } else if(audioType.equalsIgnoreCase("vlc") ||
audioType.equalsIgnoreCase("mp4")) {
            mediaAdapter = new MediaAdapter(audioType);
            mediaAdapter.play(audioType);
        } else {
            System.out.println("Invalid media type: " + audioType + " format
not supported.");
        }
    }
}
```

Adapter (class structural)

```
public class Main {  
    public static void main(String[] args) {  
        AudioPlayer audioPlayer = new AudioPlayer();  
  
        audioPlayer.play("mp3");  
        audioPlayer.play("mp4");  
        audioPlayer.play("vlc");  
        audioPlayer.play("avi");  
    }  
}
```

Output:

Playing mp3 file

Playing mp4 file

Playing vlc file

Invalid media type: avi format not supported.

- o Introduction
- o Use of design patterns
- o Detailed Discussion and Examples
 - Object Behavioral Patterns
 - Object Creational Patterns
 - Object Structural Patterns
 - Class Behavioral Patterns
 - Class Creational Patterns
 - Class Structural Patterns
- o **Summary**

Design Patterns: Discussion

- Design patterns are applicable in all stages of the OO lifecycle
 - Analysis, design, & reviews
 - Realization & documentation
 - Reuse & refactoring
- Design patterns permit design at a more abstract level
 - Treat many class/object interactions as a unit
 - Often beneficial after initial design
 - Target for class refactoring

Design Patterns: Discussion

- Variation-oriented design
 - Consider what design aspects are variable
 - Identify applicable pattern(s)
 - Vary patterns to evaluate tradeoffs
 - Repeat
- Pattern design even harder than OO design!
- Don't apply them blindly: Increased complexity and cost
- Resist branding everything as a pattern
 - Demonstrate wide applicability
 - Find at least three existing examples from code other than your own!

Benefits of Design Patterns

- Design reuse
- Uniform design vocabulary/terminology
- Enhance understanding, restructuring, & team communication
- Basis for automation
- Abstract away many unimportant details

Liabilities of Design Patterns

- Require significant, tedious & error-prone human efforts to handcraft the reuse of implementation design
- Maybe, deceptively simple uniform design vocabulary
- May limit design options
- Leave some important details unresolved

Object-Oriented Design Patterns with examples given

		<i>Purpose</i>		
		Creational	Structural	Behavioral
Scope	Class	<u>Factory Method</u>	<u>Adapter (class)</u>	Interpreter <u>Template Method</u>
	Object	Abstract Factory Builder Prototype <u>Singleton</u>	Adapter (object) Bridge <u>Composite</u> Decorator Flyweight Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento <u>Observer</u> State Strategy Visitor

■ Books

- **Design Patterns - Elements Of Reusable Object-Oriented Software**
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Analysis Patterns, Fowler; 0-201-89542-0
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Pattern Languages of Program Design
 - Vol. 1, Coplien, et al., eds., ISBN 0-201-60734-4
 - Vol. 2, Vlissides, et al., eds., 0-201-89527-7
 - Vol. 3, Martin, et al., eds., 0-201-31011-2
 - Vol. 4, Harrison, et al., eds., 0-201-43304-4
 - Vol. 5, Manolescu, et al., eds., 0-321-32194-4
- AntiPatterns, Brown, et al., 0-471-19713-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- Pattern Hatching, Vlissides, 0-201-43293-5
- Design Patterns Explained, Shalloway & Trott, 0-201-71594-5

- Articles
 - “Object-Oriented Patterns,” P. Coad; Comm. of the ACM, 9/92
 - “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92
 - “Design Patterns: Abstraction & Reuse of Object-Oriented Design,” Gamma, Helm, Johnson, Vlissides, ECOOP '93

**Thank you for
Your attention...**